Отказоустойчивость расчета с Apache ZooKeeper

Лабораторная работа 4

Вариант 8 - Программист, разработчик Frontend

Цель и результат

В этой работе я добавил отказоустойчивый координатор к расчету частот навыков из ДЗ 3. Развернул ансамбль из трех серверов ZooKeeper, проверил кворум и эфемерный узел. Затем запустил две копии координатора и приостановил лидера на 40 секунд. Запасной продолжил расчет с чекпоинта и закончил его до возобновления основного процесса.

Опыты выполнил в двух режимах: без защиты публикации и с FENCING=1. В первом случае старый лидер записал результат повторно. Во втором получатель отклонил его запрос с BadVersionError.

1 Ансамбль и проверка кворума

Использовал официальный Docker-образ zookeeper:3.9.6-jre-17. Серверам назначены myid 1, 2 и 3. У каждого свои тома /data и /datalog, поэтому данные серверов не пересекаются. Список server.N одинаковый во всех трех конфигурациях. tickTime равен 2000 мс, initLimit - 10, syncLimit - 5.

server.1=zk1:2888:3888;2181

server.2=zk2:2888:3888;2181

server.3=zk3:2888:3888;2181

Внутри контейнеров клиентский порт одинаковый, 2181. На компьютере используются разные порты: 22181, 22182 и 22183. Они доступны только через 127.0.0.1. Роли проверял командой zkServer.sh status, а доступность - настоящей клиентской операцией create, чтением созданного значения и delete.

Состояние	**zk1**	**zk2**	**zk3**	**Запись**
Все работают	follower	leader	follower	Успешно
Остановлен zk3	follower	leader	Остановлен	Успешно
Остановлены zk3 и zk2	Нет обслуживания	Остановлен	Остановлен	Тайм-аут
Оба возвращены	follower	leader	follower	Успешно

После остановки второго сервера контейнер zk1 оставался запущен, но не обслуживал обычного клиента: получен NO_QUORUM. Это проверка потери большинства, а не просто состояния контейнера. При восстановлении я дождался появления рабочих ролей у всех трех серверов и повторил запись. Выводы находятся в evidence/ensemble.json и evidence/commands.txt.

Эфемерный узел

Открыл zkCli.sh на zk2, создал эфемерный узел и проверил его из другой сессии. Затем завершил первый клиент командой quit. Повторная проверка показала, что узла больше нет.

```
create -e /hw4_ephemeral session-owned
exists= True ephemeralOwner= 144119527288012800
quit
after_session_close_exists= False
```

Сохранены полный вывод клиента zkcli-ephemeral.txt и результат независимой проверки ephemeral.txt. Закрытие сессии в этом опыте было штатным, истечение сессии после заморозки проверено отдельно на координаторах.

2 Координатор, чекпоинт и публикация

Расчет использует тот же файл data/vacancies.csv из 229388 строк, что и ДЗ 3. В предыдущей работе этот CSV был загружен в S3, здесь я подключил его точную локальную копию в контейнеры только для чтения. Отбор selected, нормализацию skills и функцию mapper импортировал из исходных common.py и mapreduce.py без изменений.

Из входа отбираются 6075 вакансий варианта 8. Словарь состоит из 20 самых частых навыков. Считаются одиночные навыки, пары и тройки. Выборка разделена на 61 чанк по 100 строк, последний содержит 75 строк. Между чанками добавлена пауза 350 мс, чтобы успеть приостановить процесс во время работы. Пауза не меняет частоты.

Обе копии координатора подключаются к ансамблю с тайм-аутом 10 секунд. Каждая пытается создать /hw4/<run>/leader как эфемерный узел. Создать его может только один процесс. После получения лидерства процесс увеличивает версию постоянного узла /epoch. Эту версию использую как номер эпохи: у первого лидера 1, у запасного 2.

Чекпоинт хранится одним JSON в /checkpoint. Он содержит номер следующего чанка, накопленные частоты, число обработанных вакансий и эмиссий, а также отпечаток входного файла, словаря и размера чанка. Номер чанка и накопленные суммы меняются атомарно. Запасной загружает их вместе и начинает со следующего чанка, поэтому уже учтенные строки повторно не прибавляются.

Сохранение чекпоинта всегда проверяет эпоху, в том числе в опыте FENCING=0. Переключатель относится к публикации результата. Старый процесс после паузы намеренно продолжает локальную работу, ошибки SessionExpiredError, ConnectionLoss и BadVersionError при сохранении чекпоинта фиксируются в журнале. Получать новую эпоху для старого расчета он не пытается.

Отдельный получатель результата

Готовый расчет отправляется по HTTP сервису receiver. Сначала он сохраняет неизменяемый JSON по SHA256. Затем одной транзакцией обновляет указатель /result и добавляет запись в /publications. В режиме защиты в эту же транзакцию включена проверка версии /epoch:

```
tx.check(root + '/epoch', version=request['epoch'])
tx.set_data(root + '/result', pointer)
tx.create(root + '/publications/item-', pointer, sequence=True)
commit(tx)
```

Так между проверкой эпохи и публикацией нет отдельного промежутка времени. Сам файл еще не означает публикацию: видимый результат определяется указателем и записью истории. Получатель выбирает политику по пространству опыта, а не доверяет флагу в запросе клиента.

3 Приостановка лидера без защиты

Запустил RUN_ID=unfenced и FENCING=0. Первым лидером стал coord-b с эпохой 1, запасным - coord-a. Дождался промежуточного чекпоинта и выполнил docker compose pause для контейнера лидера. Заморозились сам координатор, его heartbeat и дочерние процессы.

Событие	**Время по Москве**
Команда pause начата	18:08:38.597
Команда pause завершена	18:08:38.702
Запасной стал лидером, эпоха 2	18:08:50.084
Запасной опубликовал результат	18:09:12.750
Команда unpause завершена	18:09:18.973

Во время паузы в ZooKeeper был сохранен чекпоинт next_chunk=3, то есть 300 вакансий уже учтены. Запасной загрузил именно этот чекпоинт и продолжил с чанка 4. Пауза длилась 40,27 с. Время от заморозки до получения лидерства запасным составило 11,28-11,49 с.

После истечения сессии эфемерный узел первого координатора исчез. Запасной занял его, получил эпоху 2, завершил оставшиеся чанки и первым опубликовал результат. Старый лидер был возобновлен только после этой публикации.

coord-a: записал результат, epoch=2, pointer_version=1

coord-b: записал результат, epoch=1, pointer_version=2

Старый лидер также завершил локальный расчет и в 18:09:40.495 отправил его получателю. Без проверки эпохи запрос был принят. В /publications появились две записи, а версия /result стала 2. Итоговый указатель принадлежит coord-b, то есть устаревшему лидеру.

Два результата имеют одинаковые частоты и SHA256, поэтому содержимое вычисления корректно, но публикация выполнена дважды. Это видно одновременно в журналах двух координаторов, журнале получателя и снимке состояния unfenced.json. Одинаковые JSON хранятся одним файлом по хешу, две публикации отражены двумя записями истории.

4 Приостановка лидера с защитой по эпохе

Повторил опыт в отдельном пространстве RUN_ID=fenced с FENCING=1. Лидером стал coord-b с эпохой 1, запасным - coord-a. Порядок действий сохранил: промежуточный чекпоинт, заморозка лидера, завершение работы запасным, затем возобновление старого процесса.

Событие	**Время по Москве**
Команда pause начата	19:23:43.452
Команда pause завершена	19:23:43.539
Запасной стал лидером, эпоха 2	19:23:54.133
Запасной опубликовал результат	19:24:16.701
Команда unpause завершена	19:24:23.797

На момент паузы чекпоинт содержал next_chunk=3. Запасной восстановил этот номер и накопленные счетчики. Время смены лидера составило 10,48-10,69 с, общая пауза - 40,258 с.

coord-a: записал результат, epoch=2, pointer_version=1

coord-b: publication_rejected, error=BadVersionError

После возобновления старый лидер отправил результат с эпохой 1. Получатель сравнил ее с текущей версией /epoch, равной 2. Проверка не прошла, и вся транзакция была отклонена. Указатель остался от coord-a, его версия осталась 1, в истории осталась одна публикация.

Здесь проверяется версия узла эпохи, а не только версия указателя результата. Поэтому устаревшая запись была бы запрещена и до первой публикации нового лидера, сразу после увеличения эпохи. Проверка выполняется в атомарной транзакции на стороне получателя.

Полная сверка с ДЗ 3

Показатель	**ДЗ 3**	**Без защиты**	**С защитой**
Вакансии	6075	6075	6075
Эмиссии навыков	19307	19307	19307
Эмиссии пар	41058	41058	41058
Эмиссии троек	58679	58679	58679
Уникальные навыки	20	20	20
Уникальные пары	190	190	190
Уникальные тройки	1120	1120	1120

Кроме контрольных чисел, сравнил каждую запись (навыки, размер сочетания, частота) с mr_optimized_20_1.json и duckdb_20_1.json. Сравнение прошло у обоих координаторов в каждом опыте и затем отдельно в verify_evidence.py. Все 1330 записей совпали. Результаты двух опытов имеют одинаковый SHA256.

5 Ответы на вопросы

Почему три сервера, а не два или четыре?

Для подтверждения изменений ZooKeeper нужно большинство участников. Если серверов два, большинство равно двум: после одного отказа остается только один и работа останавливается. Для трех серверов большинство равно двум, поэтому один отказ допустим. Для четырех большинство равно трем, и допустим все тот же один отказ. Четвертый голосующий сервер требует ресурсов, но в этой конфигурации не увеличивает число переносимых отказов. Чтобы пережить два отказа, нужны пять участников и большинство из трех.

Большинство также не дает двум изолированным частям ансамбля независимо подтверждать противоречащие изменения: любые два большинства пересекаются. Поэтому потеря кворума приводит к остановке обслуживания, как в опыте с единственным оставшимся сервером.

Почему эпоху проверяет получатель записи, а не ZooKeeper?

ZooKeeper отслеживает сессии и помогает выбрать нового лидера. Он не может остановить приостановленный процесс и не управляет всеми действиями приложения. Старый координатор после возобновления еще хранит в памяти свою роль, промежуточные данные и адрес получателя. Если получатель принимает любой запрос, этот процесс способен повторно записать результат, хотя его сессия уже истекла.

Поэтому правило допуска записи должно действовать там, где запись становится видимой. В моей реализации это receiver: он требует актуальную эпоху и включает проверку в транзакцию публикации. ZooKeeper здесь выполняет атомарный check по запросу получателя, поскольку указатель тоже хранится в ZooKeeper. Сам по себе сервис не добавляет эту проверку за приложение. Для записи в отдельную базу или объектное хранилище аналогичное правило пришлось бы реализовать на стороне соответствующего получателя.

Как измерял время

Перед командой pause и после ее завершения сценарий считывает time.monotonic() в контейнере получателя. Координаторы используют тот же монотонный счетчик Linux VM Docker. Фактический момент заморозки находится между этими двумя отметками, поэтому время смены лидера приведено интервалом. Конец интервала измерения - событие became_leader у запасного, после получения эпохи. Время окончания расчета в него не входит. Тайм-аут сессии 10 секунд не означает переключение ровно через 10 секунд: влияют момент последнего heartbeat и обработка истечения сессии сервером.

Что проверено

Проверены отказы отдельных серверов ZooKeeper и заморозка процесса координатора. Все контейнеры находятся на одном компьютере, поэтому отказ физического хоста не моделировался. Получатель результата работает одним процессом, его собственный отказ не проверялся. Защита по эпохе устраняет публикацию устаревшего лидера, но сама по себе не дает общую гарантию exactly-once для произвольных повторов HTTP. В подготовительных попытках уточнил измерение чекпоинта и обработку истечения сессии их журналы сохранены отдельно в evidence/preliminary-*.

6 Воспроизведение и состав сдачи

Все команды выполняются из корня репозитория при запущенном Docker Desktop. Для сценария на компьютере нужен Python 3, библиотеки приложения устанавливаются в образ. Для нового полного запуска нужны пустые каталоги журналов и новое состояние учебного ансамбля. Сохраненные материалы сначала следует скопировать отдельно.

```
docker compose -f hw4/compose.yaml up -d --build zk1 zk2 zk3 receiver
python3 hw4/run_experiments.py
python3 hw4/verify_evidence.py
```

Сценарий сам задает RUN_ID и FENCING, запускает два координатора, выбирает фактического лидера по журналу, выполняет pause/unpause и проверяет итоговые версии указателей. После восстановления ансамбля он ждет рабочие роли, а не полагается только на фиксированную задержку. Для повторения только опытов координаторов после уже выполненной проверки ансамбля предусмотрен параметр --coordinators-only.

```
ALL ACCEPTANCE CHECKS PASSED
PASS: quorum, ephemeral session, failover, checkpoint, fencing,
full HW3 equivalence, artifact SHA256
```

Первая строка - итог сценария реальных экспериментов, вторая - итог независимой проверки сохраненных материалов. Подробный вывод второй проверки находится в evidence/verification.txt.